

Smart Blind Assistant

Documentation technique — Architecture & Décisions de conception

1. Vue d'ensemble du projet

Smart Blind Assistant est un assistant pour malvoyants combinant Computer Vision et NLP. Il repose sur un backend Python (FastAPI) et utilise YOLOv8 pour la détection d'objets. Trois modules CV principaux cohabitent : détection d'obstacles, recherche d'objet précis, et localisation de pièce.

2. Structure du projet

Fichier / Dossier	Rôle
cv/core/model_loader.py	Singleton — charge le modèle YOLO une seule fois
cv/detector.py	Détecte les obstacles proches + retourne position et distance
cv/finder.py	Cherche un objet précis parmi les détections
cv/localizer.py	Infère la pièce courante selon les objets détectés
cv/train.py	Fine-tuning du modèle (utilisé une seule fois)
cv/models/yolov8n.pt	Modèle YOLO de base pré-entraîné
cv/models/best.pt	Modèle fine-tuné (remplace yolov8n après entraînement)
navigation/	Graphe de navigation + instructions (pathfinder, graph)
nlp/	Traitement des intentions utilisateur
safety/sos.py	Gestion des situations d'urgence
test/	Tests unitaires de chaque module
main.py	Point d'entrée FastAPI

3. Principe du model_loader (Singleton)

Sans singleton, si les 3 modules (detector, finder, localizer) sont utilisés simultanément, YOLO serait chargé 3 fois en mémoire. Le singleton garantit un seul chargement :

```
_model = None

def get_model(path='cv/models/yolov8n.pt'):
    global _model
    if _model is None:
        _model = YOLO(path)
    return _model
```

4. detector.py — Logique de détection

4.1 Position horizontale (gauche / centre / droite)

YOLO retourne les coordonnées x1, x2 de la bounding box. On calcule le centre horizontal $cx = (x1+x2)/2$, puis on divise l'image en 3 zones égales :

Zone	Condition	Label
Gauche	$cx < \text{img_width} / 3$	"gauche"
Centre	$\text{img_width}/3 \leq cx < 2*\text{img_width}/3$	"centre"
Droite	$cx \geq 2*\text{img_width} / 3$	"droite"

4.2 Distance — Option 1 : Taille de référence par classe

Problème identifié : un ratio brut bbox/image favorise les grands objets. Un vase proche semblerait loin comparé à un lit lointain. Solution : normaliser le ratio par une taille de référence propre à chaque classe :

```
ratio = bbox_area / img_area
ref = TAILLE_REF.get(cls_name, 0.08)
ratio_normalise = ratio / ref

si ratio_normalise > 2.0 → 'tres proche'
si ratio_normalise > 0.8 → 'proche'
sinon → 'loin'
```

Note : Cette approche est une approximation valable pour un usage temps réel. Une amélioration future possible est l'intégration de MiDaS (depth estimation) pour une mesure de profondeur réelle.

5. finder.py et localizer.py

5.1 finder.py — Recherche d'un objet précis

Même modèle YOLO partagé via `get_model()`. La logique filtre les détections pour ne retourner que la classe demandée par l'utilisateur (ex: 'trouve mon téléphone'). Retourne la position et la distance de l'objet cible.

5.2 localizer.py — Reconnaissance de pièce

Approche par règles basées sur les détections YOLO. Chaque pièce est définie par un ensemble d'objets signature. On calcule un score par pièce = nombre d'objets détectés correspondant à la signature :

Pièce	Objets signature	Indicateur dominant
Salon	canapé, TV, table basse, télécommande	canapé + TV = très forte probabilité
Cuisine	réfrigérateur, four, évier, bouteille	frigo + évier + four = quasi certain
Chambre	lit, armoire, lampe, miroir	lit = signal dominant
Salle de bain	lavabo, miroir, toilette	lavabo + miroir + toilette
Couloir	porte, escalier (absence de meubles)	portes + pas de mobilier
Bureau	laptop, clavier, écran, chaise bureau	laptop + écran = bureau

6. Stratégie de Fine-tuning

YOLOv8 est pré-entraîné sur COCO (80 classes). La majorité des objets nécessaires sont déjà couverts. Un fine-tuning léger est prévu pour les classes manquantes :

Objet manquant	Raison
robinet	absent de COCO
prise de courant	absent de COCO
flamme	absent de COCO
escalier	dans COCO mais peu fiable
douche	absent de COCO
armoire / closet	approximatif dans COCO
serviette	absent de COCO
table de nuit	absent de COCO

Plan : collecter 100-200 images par classe manquante, fine-tuner sur yolov8n.pt, sauvegarder best.pt. Seul `cv/core/model_loader.py` doit être mis à jour (changer le chemin du modèle). Tout le reste du code reste inchangé.

7. Plan de développement

Phase	Tâches	Statut
Phase 1 Code de base	<code>model_loader.py</code> , <code>detector.py</code> , <code>finder.py</code> , <code>localizer.py</code> Tests unitaires	En cours
Phase 2 Fine-tuning	Collecte dataset, annotation, entraînement Remplacement de yolov8n.pt par best.pt	A venir
Phase 3 Intégration	Connexion avec NLP (<code>intent.py</code>) API FastAPI (<code>main.py</code>)	A venir
Phase 4 Optimisation	MiDaS pour depth estimation (optionnel) Tests en conditions réelles	A venir

8. Amélioration future — Depth Estimation

Approche	Précision distance	Vitesse	Classes	Complexité
Option 1 (taille ref)	Faible	Tres rapide	Toutes	Simple
MediaPipe Objectron	Moyenne	Moyen	~9 seulement	Moyen
MiDaS	Bonne	Lent	Toutes	Moyen

Decision : Option 1 retenue pour le court terme. MiDaS envisageable en Phase 4 si la précision de distance s'avère insuffisante en conditions réelles. MediaPipe Objectron écarté car trop peu de classes supportées.